



UNIVERZITET U SARAJEVU
ELEKTROTEHNIČKI FAKULTET
ODSJEK ZA AUTOMATIKU I ELEKTRONIKU

Laboratorijska vježba 3

Praktikum iz logičkog dizajna i upravljanja zasnovan na FPGA razvojnom sistemu Gowin 9k

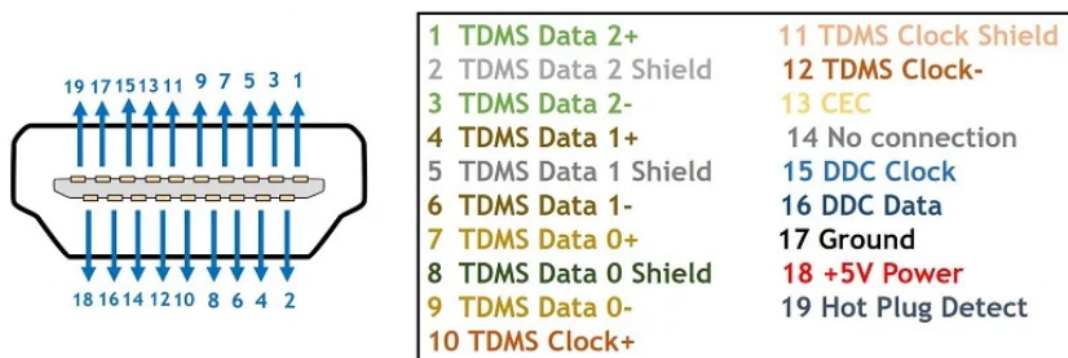
Tarik Ćatić

U ovoj vježbi student se upoznaje sa HDMI komunikacijom i na FPGA razvojnom sistemu Tang Nano 9K implementira jednostavnu interaktivnu igru sa pokretnom lopticom i preprekom, uz signalizaciju sudara pomoću LED dioda.

Teorijski uvod – HDMI komunikacija

HDMI (engl.: *High-Definition Multimedia Interface*) je standardizovani protokol za prijenos nekompresovanog digitalnog video i audio signala između uređaja, prvi put predstavljen 2002. godine, koji je od tada postao dominantan način povezivanja televizora, monitora i konzola sa izvorima signala poput računara i FPGA razvojnih sistema [1]. Fizički, standardni HDMI konektor tipa A sadrži 19 pinova: tri para pinova nose video podatke (po jedan par za crvenu, zelenu i plavu komponentu boje), jedan par nosi referentni takt, dok preostali pinovi služe za pomoćne funkcije poput DDC/EDID komunikacije (razmjena podataka o mogućnostima monitora) i CEC protokola [1]. Raspored pinova prikazan je na slici 1.

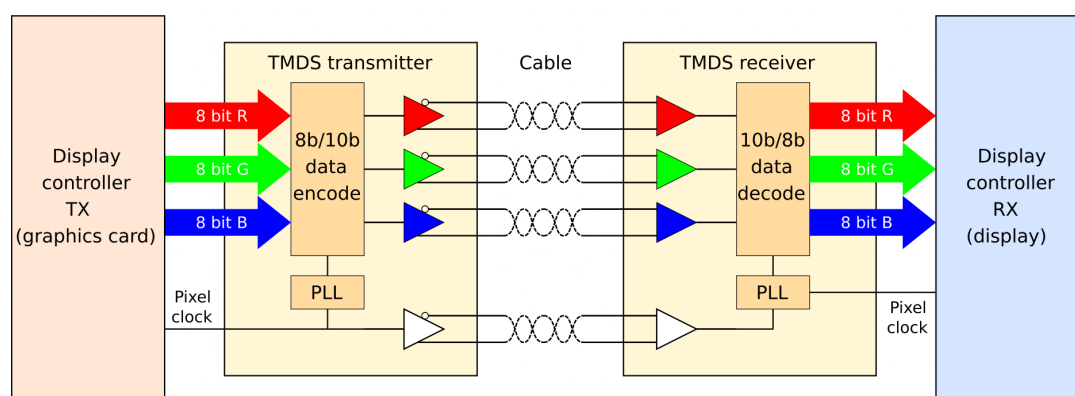
Type A HDMI connector pinout



www.eTechnophiles.com

Slika 1: Raspored pinova HDMI konektora tipa A.

Prijenos podataka kroz sva četiri diferencijalna para zasniva se na TMDS (engl.: *Transition-Minimized Differential Signaling*) tehnologiji. Umjesto da se signal šalje kao jedna linija u odnosu na masu, TMDS koristi dvije komplementarne linije (diferencijalni par) po kojima se šalje isti signal u suprotnim fazama – prijemnik potom očitava razliku napona između te dvije linije, čime se značajno smanjuje osjetljivost na elektromagnetne smetnje u odnosu na jednos-truki (engl.: *single-ended*) prijenos signala [2]. Prikaz principa diferencijalnog prijenosa dat je na slici 2.



Slika 2: Princip TMDS diferencijalnog prijenosa signala.

Dodatno, prije samog slanja, svakih 8 bita video podataka (po jednoj boji, po jednom pikselu) enkodira se u 10-bitnu TMDS riječ, pomoću algoritma koji ima dva cilja: minimizirati broj promjena logičkog stanja na liniji (radi smanjenja elektromagnetnih smetnji) i istovremeno održati dovoljan broj tranzicija da bi prijemnik mogao pouzdano rekonstruisati takt iz same podatkovne linije (engl.: *clock recovery*) [2]. TMDS takt (jedan od četiri diferencijalna para) odgovara frekvenciji piksela (engl.: *pixel clock*) i služi prijemniku kao vremenska referenca za očitavanje preostala tri podatkovna para.

Period slanja jednog kompletnog videa okvira dijeli se na tri vrste perioda: period aktivnog video signala (kada se prenose stvarni pikseli slike), period sinhronizacije (kada se prenose horizontalni/vertikalni sinhronizacioni impulsi, van vidljivog dijela ekrana) i, opcionalno, period pomoćnih podataka (engl.: *data island*) za prijenos audio signala i dodatnih informacionih

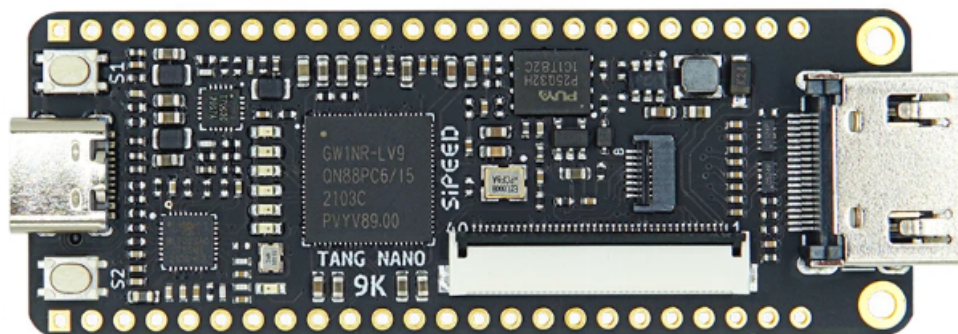
paketa [2]. Za potrebe ove laboratorijske vježbe, koristi se isključivo video i sinhronizacioni period – audio prijenos nije implementiran.

Struktura startnog projekta

Za realizaciju HDMI komunikacije korišten je pripremljeni startni projekat, prilagođen tačno za Tang Nano 9K razvojni sistem (GW1NR-9C čip), čija se struktura sastoji od sljedećih fajlova:

- `top.sv` – najviši nivo hijerarhije. Sadrži logiku za generisanje boje svakog piksela na osnovu njegovih X i Y koordinata (koje dobija od HDMI kontrolera), te instancira glavni HDMI modul.
- `hdmi_pkg.sv` – SystemVerilog paket sa zajedničkim tipovima podataka i parametrima vremenskog rasporeda ekrana (broj piksela po redu/koloni, pozicija i trajanje sinhronizacionih impulsa) za odabranu rezoluciju.
- `hdmi_backend.sv` – centralni modul koji generiše taktove (preko PLL-a), broji trenutnu X, Y poziciju piksela, generiše horizontalni i vertikalni sinhronizacioni signal, te prosljeđuje pikselske podatke na TMDS enkodiranje i serializaciju.
- `hdmi_tdms_enc.sv` – implementacija TMDS 8b/10b algoritma opisanog u prethodnoj sekciji, za svaki od tri kolor kanala posebno.
- `src/fpgatech_lib/GOWIN/` – folder sa tehnologijom-specifičnim omotačima (*wrapper* modulima) oko Gowin primitiva: `fpga_pll.sv` (generisanje piksel takta iz 27 MHz oscilatora ploče), `fpga_oser10.sv` (serializacija 10-bitne TMDS riječi), i `fpga_olvds.sv` (izlazni LVDS baferi koji fizički pokreću diferencijalne parove).
- `constraints/pinout.cst` – fizičko mapiranje signala na konkretne pinove FPGA čipa.

Razvojni sistem korišten u ovoj vježbi prikazan je na slici 3.



Slika 3: Tang Nano 9K razvojni sistem sa HDMI konektorom.

Prilagođavanje HDMI predloška odabranom FPGA uređaju i monitoru

Startni HDMI projekat je generički napisan tako da se lako prenosi (*portuje*) na različite FPGA platforme, ali prije upotrebe ga je potrebno prilagoditi konkretnoj kombinaciji FPGA uređaja i monitora koji se koriste. U ovom radu, to su:

- **FPGA uređaj:** Tang Nano 9K razvojni sistem (proizvođač Sipeed), sa GW1NR-9C čipom, tačnog part numbera GW1NR-LV9QN88PC6/I5, taktovan eksternim oscilatorom od 27 MHz;
- **Monitor:** standardni HDMI monitor rezolucije 1280×720 piksela pri frekvenciji osvježavanja od 60 Hz (*VideoMode=4* prema HDMI/CEA-861 standardu).

Prilagođavanje predloška ovoj kombinaciji uređaja obuhvata sljedeće korake:

1. **Odabir FPGA uređaja u projektu** – u postavkama Gowin EDA projekta potrebno je odabrati tačan model čipa (GW1NR-9C, part number GW1NR-LV9QN88PC6/I5), jer se prema tom modelu generišu parametri PLL bloka u fajlu `fpga_pll sv`.
2. **Parametri rezolucije ekrana** – u fajlu `hdmi_pkg sv` definisani su parametri vremenskog rasporeda (broj piksela po redu i koloni, pozicija i trajanje sinhronizacionih impulsa) tačno za rezoluciju $1280 \times 720@60\text{Hz}$. Za ovu kombinaciju uređaja ovi parametri se koriste bez izmjene; ukoliko bi se koristio monitor druge rezolucije, ove vrijednosti bi bilo potrebno zamijeniti odgovarajućim standardnim vrijednostima za tu rezoluciju.

3. **Podešavanje PLL bloka** – fajl `fpga_pll . sv` iz ulaznog takta od 27 MHz (frekvencija oscilatora na Tang Nano 9K pločici) generiše dva izlazna takta potrebna za odabranu rezoluciju: piksel takt od 74.25 MHz i $5\times$ serijski takt od 371.25 MHz. Množitelji/djelitelji PLL-a (`IDIV_SEL`, `FBDIV_SEL`, `ODIV_SEL`) su izračunati tačno za ovu kombinaciju ulaznog takta i ciljane rezolucije, i već su podešeni za GW1NR-9C čip.
4. **Fizička dodjela pinova** – fajl `pinout . cst` sadrži mapiranje HDMI signala (takt i tri podatkovna diferencijalna para) na tačne fizičke pinove Tang Nano 9K pločice na kojima je fizički izveden HDMI konektor, kao i mapiranje dva tastera i eksternog oscilatora. Za drugu razvojnu pločicu ove lokacije bi bilo potrebno prilagoditi prema šemi te pločice.
5. **Sinteza, generisanje bitstream-a i programiranje** – nakon što su sva prethodna podešavanja usklađena sa korištenim uređajima, projekat se sintetizuje na uobičajen način, a generisani bitstream se prenosi na Tang Nano 9K putem USB-C porta.

Implementacija interaktivne igre – pokretna loptica

Na osnovu prilagođenog predloška, implementirana je jednostavna interaktivna igra: crvena loptica prikazana na ekranu, čija se vertikalna pozicija mijenja u zavisnosti od toga koje je dugme na razvojnoj pločici pritisnuto.

Crtanje loptice

Boja svakog piksela određuje se na osnovu njegove udaljenosti od centra loptice. Piksel na poziciji (x, y) boji se crveno ako se nalazi unutar kruga poluprečnika r centriranog u tački (x_0, y_0) , odnosno ako vrijedi:

$$(x - x_0)^2 + (y - y_0)^2 \leq r^2 \quad (1)$$

Horizontalna koordinata centra x_0 je fiksna i postavljena na 640 (sredina ekrana rezolucije 1280×720), dok se vertikalna koordinata y_0 čuva u registru `brojac` i mijenja pritiskom na dugmad. Prag $r^2 = 250$ odgovara poluprečniku od približno 16 piksela.

Kod 1 Uslov za crtanje loptice unutar `_my_game` bloka.

```
1 always_comb begin: _my_game
2     // Crvena loptica poluprecnika ~sqrt(250)=15.8px, centrirana u
3     // Van kruga, pozadina je jednoboja plava.
4     if ((x-640)*(x-640)+(y-brojac)*(y-brojac)<=250) begin
5         pix.R = '1;
6         pix.G = '0;
7         pix.B = '0;
8     end
9     else begin
10        pix.R = '0;
11        pix.G = '0;
12        pix.B = '1;
13    end
```

```
14 end: _my_game
```

Upravljanje pozicijom loptice

Pozicija loptice se ažurira unutar istog sekvencijalnog bloka koji generiše i signal za LED diodu, koji se aktivira jednom u svakih MFRAM_CNT_MAX (30) prikazanih okvira – što odgovara ažuriranju otprilike svakih pola sekunde, dovoljno sporo da je kretanje jasno vidljivo. Dugme butnL_n pomjera lopticu prema gore, a butnR_n prema dole (oba dugmeta su aktivna-niska).

Kretanje je ograničeno na opseg $[BALL_Y_MIN, BALL_Y_MAX] = [20, 700]$, čime loptica (poluprečnika ≈ 16 piksela) uvijek ostaje potpuno unutar vidljivog dijela ekrana (0–719), sa mekim zaustavljanjem na gornjoj i donjoj granici. Početna pozicija nakon reseta je $y_0 = 360$, tačno na vertikalnoj sredini ekrana.

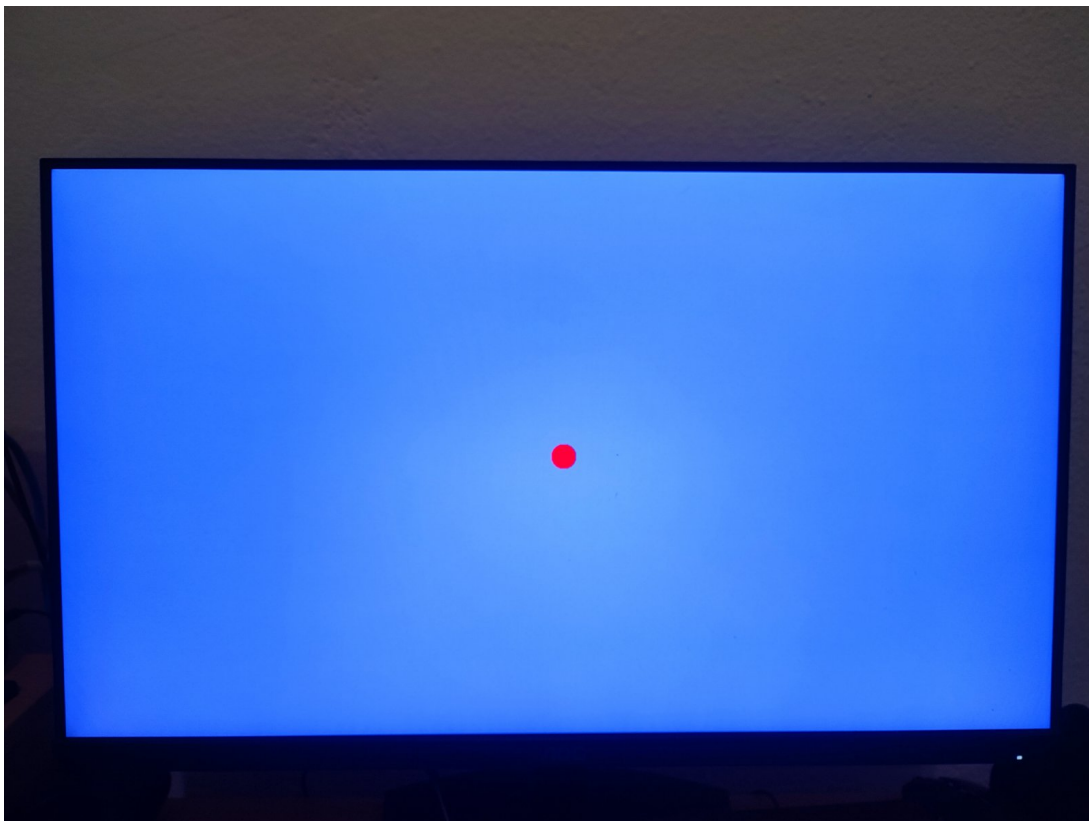
Kod 2 Logika resetovanja i pomjeranja loptice.

```
1  localparam bus12_t BALL_Y_MIN  = 12'd20;    // gornja granica
   kretanja
2  localparam bus12_t BALL_Y_MAX  = 12'd700;    // donja granica
   kretanja
3  localparam bus12_t BALL_Y_INIT = 12'd360;    // pocetna pozicija -
   sredina ekrana
4  localparam bus12_t BALL_STEP   = 12'd10;     // korak pomjeranja po
   jednom updateu
5
6  bus12_t brojac;
7
8  always_ff @(negedge srst_n or posedge clk_pix) begin: _led_mon
9      if (srst_n == LO) begin
10         led_n  <= HI;
11         brojac <= BALL_Y_INIT;
12     end
13     else begin
14         if (mfram_done == HI) begin
15             led_n <= ~led_n;
16
17             if (butnL_n == LO) begin
18                 if (brojac > BALL_Y_MIN + BALL_STEP)
19                     brojac <= brojac - BALL_STEP;    // gore
20                 else
21                     brojac <= BALL_Y_MIN;
22             end
23             else if (butnR_n == LO) begin
24                 if (brojac < BALL_Y_MAX - BALL_STEP)
25                     brojac <= brojac + BALL_STEP;    // dole
26                 else
27                     brojac <= BALL_Y_MAX;
28             end
29         end
30     end
31 end
```

```
29         end
30     end
31 end: _led_mon
```

Rezultat

Pritiskom na dugme S1 (butnL_n) loptica se pomjera prema gore, a pritiskom na S2 (butnR_n) prema dole, sa glatkim zaustavljanjem na gornjoj i donjoj granici ekrana. Rezultat prikazan na monitoru dat je na slici 4.



Slika 4: Crvena loptica prikazana na monitoru putem HDMI izlaza.

Dodavanje pokretne prepreke

Nakon osnovne implementacije loptice čija se vertikalna pozicija upravlja dugmadima, projekat je proširen dodavanjem pokretne prepreke koju je potrebno izbjegavati – koncept sličan poznatoj igri *Flappy Bird*, prilagođen tako da je horizontalna pozicija loptice fiksna, a igrač je pomjera isključivo vertikalno kako bi prošao kroz otvor na prepreci koja se kreće zdesna nalijevo.

Struktura prepreke

Prepreka je predstavljena kao vertikalni zid širine `OBST_WIDTH` (40 piksela), sa pravougaonim otvorom (prorezom) visine `GAP_HEIGHT` (200 piksela). Umjesto praćenja više prepreka istovremeno, korištena je najjednostavnija moguća šema: u svakom trenutku postoji tačno jedna

prepreka na ekranu, predstavljena registrom `obst_x` koji čuva njenu trenutnu horizontalnu poziciju.

Kod 3 Parametri i signali prepreke.

```
1 localparam bus12_t OBST_WIDTH  = 12'd40;
2 localparam bus12_t OBST_SPEED  = 12'd6;
3 localparam bus12_t GAP_HEIGHT  = 12'd200;
4
5 bus12_t      obst_x;    // trenutna X pozicija lijeve ivice prepreke
6 bus12_t      gap_y;     // trenutna Y pozicija vrha proreza
7 logic [1:0] gap_sel;    // redni broj trenutne pozicije proreza (0-3)
```

Kretanje prepreke

Pozicija prepreke se ažurira jednom po svakom prikazanom okviru (signal `frame_done`, 60 puta u sekundi), pri čemu se `obst_x` smanjuje za `OBST_SPEED` piksela. Kada prepreka izađe van lijeve ivice ekrana, umjesto da se dozvoli neograničeno opadanje registra (što bi, s obzirom na to da je `obst_x` bez predznaka, dovelo do nekontrolisanog prekoračenja), eksplicitno se provjerava granica i prepreka se vraća na početnu poziciju van desne ivice ekrana:

Kod 4 Sekvencijalna logika kretanja prepreke.

```
1 always_ff @(negedge srst_n or posedge clk_pix) begin: _obstacle_mov
2     if (srst_n == LO) begin
3         obst_x  <= 12'(HSCREEN) + OBST_WIDTH; // pocinje van desne
           // ivice ekrana
4         gap_y   <= gap_preset(2'd0);
5         gap_sel <= 2'd0;
6     end
7     else if (frame_done == HI) begin
8         if (obst_x <= OBST_SPEED) begin
9             gap_sel <= gap_sel + 2'd1;
10            gap_y   <= gap_preset(gap_sel + 2'd1);
11            obst_x  <= 12'(HSCREEN) + OBST_WIDTH;
12        end
13        else begin
14            obst_x <= obst_x - OBST_SPEED;
15        end
16    end
17 end: _obstacle_mov
```

Pozicija proreza

Kako bi svaki prolazak prepreke djelovao drugačije, umjesto stvarnog generatora slučajnih brojeva (koji bi unio nepotrebnu složenost za ovako jednostavnu igru), korištene su četiri unaprijed

pripremljene pozicije proreza, kroz koje se ciklično prolazi pomoću brojača `gap_sel` svaki put kada prepreka krene ispočetka:

Kod 5 Funkcija za odabir pozicije proreza.

```
1 function automatic bus12_t gap_preset(input logic [1:0] sel);
2     case (sel)
3         2'd0:      gap_preset = 12'd60;
4         2'd1:      gap_preset = 12'd220;
5         2'd2:      gap_preset = 12'd380;
6         default:   gap_preset = 12'd500;
7     endcase
8 endfunction
```

Is crtavanje prepreke

Prepreka se is crtava zelenom bojom na svim pikselima koji se nalaze unutar njene širine, osim na mjestu proreza. Loptica ima prioritet is crtavanja – ukoliko se piksel nalazi i unutar loptice i unutar prepreke, prikazuje se boja loptice, čime igrač uvijek jasno vidi tačnu poziciju loptice u odnosu na prepreku:

Kod 6 Prošireni `_my_game` blok sa is crtavanjem prepreke.

```
1 always_comb begin: _my_game
2     obst_here = (x >= obst_x) && (x < obst_x + OBST_WIDTH);
3     in_gap     = (y >= gap_y) && (y < gap_y + GAP_HEIGHT);
4
5     if ((x-BALL_X)*(x-BALL_X)+(y-brojac)*(y-brojac) <= 250) begin
6         pix.R = '1; // loptica - crvena
7         pix.G = '0;
8         pix.B = '0;
9     end
10    else if (obst_here && !in_gap) begin
11        pix.R = '0; // prepreka - zelena
12        pix.G = '1;
13        pix.B = '0;
14    end
15    else begin
16        pix.R = '0; // pozadina - plava
17        pix.G = '0;
18        pix.B = '1;
19    end
20 end: _my_game
```

Detekcija sudara

Umjesto piksel-po-piksel provjere preklapanja kružnice loptice i pravougaonika prepreke (što bi zahtijevalo znatno složeniju logiku), korištena je pojednostavljena provjera na nivou intervala. Pošto je horizontalna pozicija loptice fiksna ($x_0 = 640$, poluprečnik ≈ 16 piksela), do sudara dolazi kada se prepreka horizontalno preklapa sa opsegom koji zauzima loptica, a loptica pritom nije u potpunosti unutar proreza:

Kod 7 Kombinaciona detekcija sudara.

```
1 always_comb begin: _collision_check
2     hit = (obst_x <= BALL_X + BALL_R)
3         & (obst_x + OBST_WIDTH > BALL_X - BALL_R)
4         & ~( (brojac >= gap_y + BALL_R) & (brojac + BALL_R <= gap_y +
5             GAP_HEIGHT) );
6 end: _collision_check
```

Rezultat sudara (signal `hit`) koristi se za signalizaciju korisniku putem posebne LED diode, opisano u narednoj sekciji.

Rezultat

Kombinacijom pomjeranja loptice (opisanog u prethodnoj sekciji) i pokretne prepreke sa slučajno raspoređenim prorezima, dobijena je jednostavna interaktivna igra u kojoj igrač dugmadima izbjegava sudar sa preprekama koje neprekidno pristižu sa desne strane ekrana.

Signalizacija sudara pomoću dvije nezavisne LED diode

Razvojna pločica raspolaže sa šest ugrađenih LED dioda (pinovi 10, 11, 13, 14, 15 i 16), od kojih je startni projekat koristio samo jednu (pin 11) za prikaz opšte aktivnosti sistema. Kako bi se jasno razdvojile dvije nezavisne informacije – da li sistem uopšte radi, i da li je trenutno došlo do sudara loptice sa preprekom – korištene su dvije odvojene diode, svaka zadužena za tačno jednu namjenu.

- `led_n` (pin 11) – treperi ravnomjerno jednom u svakih `MFRAM_CNT_MAX` (30) prikazanih okvira, nezavisno od bilo kakvog događaja u igri. Služi kao pokazatelj da sistem radi i da je takt aktivan.
- `led_hit_n` (pin 10) – upaljena isključivo onoliko dugo koliko traje preklapanje loptice i prepreke, računato direktno iz signala `hit` opisanog u prethodnoj sekciji.

Fizičko mapiranje novog signala dodaje se u fajl `pinout.cst`:

Kod 8 Dodatak u `pinout.cst` za drugu LED diodu.

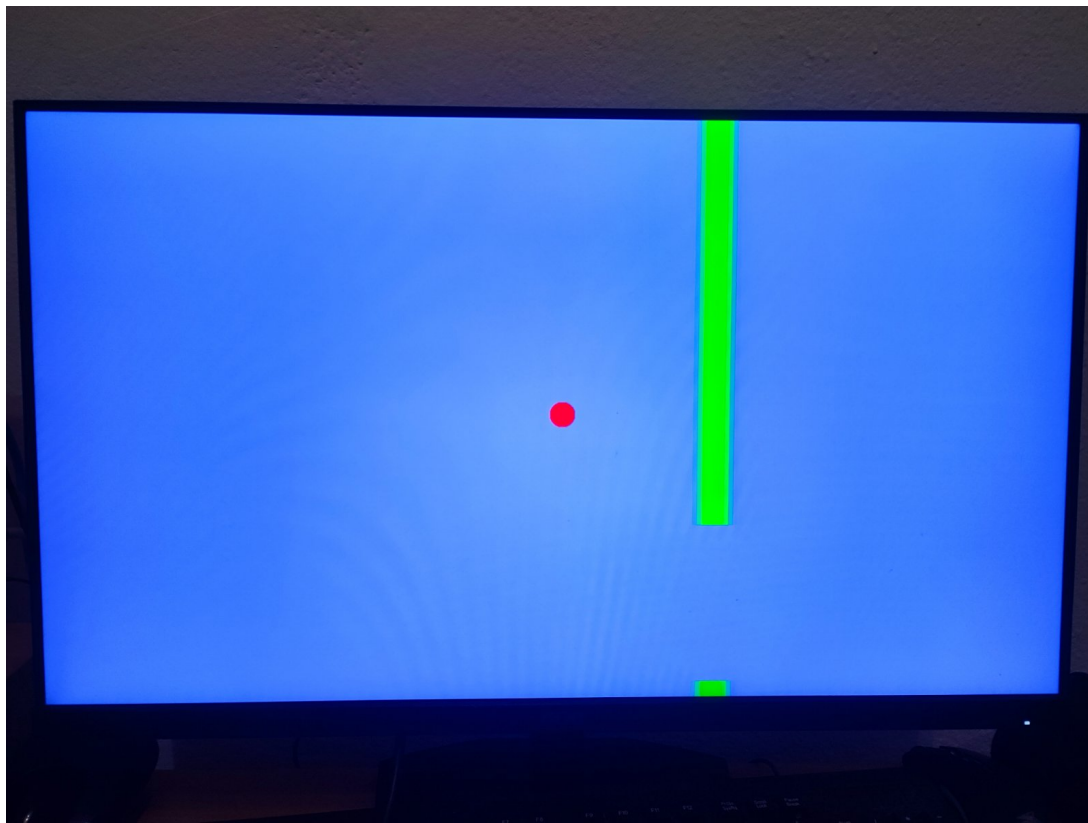
```
1 IO_LOC    "led_hit_n" 10;
2 IO_PORT   "led_hit_n" PULL_MODE=UP DRIVE=8;
```

U SystemVerilog kodu, svaka dioda se upravlja u svom vlastitom, potpuno nezavisnom sekvencijalnom bloku, čime se izbjegava bilo kakvo međusobno ometanje njihovog ponašanja:

Kod 9 Nezavisno upravljanje dvjema LED diodama.

```
1  //-----
2  // Pomjeranje loptice dugmadima. LED (led_n) treperi ravnomjerno
3  // jednom u MFRAM_CNT_MAX prikazanih okvira, nezavisno od sudara.
4  //-----
5  always_ff @(negedge srst_n or posedge clk_pix) begin: _led_mon
6      if (srst_n == LO) begin
7          led_n <= HI;
8          brojac <= BALL_Y_INIT;
9      end
10     else begin
11         if (mfram_done == HI) begin
12             led_n <= ~led_n;
13         end
14
15         if (frame_done == HI) begin
16             if (butnL_n == LO) begin
17                 if (brojac > BALL_Y_MIN + BALL_STEP)
18                     brojac <= brojac - BALL_STEP;
19                 else
20                     brojac <= BALL_Y_MIN;
21             end
22             else if (butnR_n == LO) begin
23                 if (brojac < BALL_Y_MAX - BALL_STEP)
24                     brojac <= brojac + BALL_STEP;
25                 else
26                     brojac <= BALL_Y_MAX;
27             end
28         end
29     end
30 end: _led_mon
31
32 //-----
33 // Posebna LED (led_hit_n) isključivo za signalizaciju sudara -
34 // upaljena tačno onoliko dugo koliko traje preklapanje loptice
35 // i prepreke, potpuno nezavisno od led_n treperenja.
36 //-----
37 always_ff @(negedge srst_n or posedge clk_pix) begin: _hit_led
38     if (srst_n == LO) begin
39         led_hit_n <= HI;
40     end
41     else begin
42         led_hit_n <= hit ? LO : HI;
43     end
44 end: _hit_led
```

Na slici 5 se nalazi prikaz kako izgleda interaktivna igrica nakon ubacivanja prepreke.



Slika 5: Crvena loptica i prepreka prikazani na monitoru putem HDMI izlaza.

Literatura

- [1] DisplayModule, “Introduction to hdmi protocol”, dostupno na: <https://www.displaymodule.com/blogs/knowledge/introduction-to-hdmi-protocol> 2023.
- [2] fpga4fun.com, “Hdmi – fpga to hdmi interfacing”, dostupno na: <https://www.fpga4fun.com/HDMI.html> 2023.